演習1

```
import matplotlib.pyplot as plt
import numpy as np
from scipy.integrate import solve_ivp

def ozbudak_model( t , init , *param ):
    x = init[ 0 ]
    y = init[ 1 ]

    [ alpha , beta , rho , x_out ] = param

    dxdt = -x + alpha * x_out + beta * y
    dydt = (x ** 2) / (rho + (x ** 2)) - y

    return [ dxdt , dydt ]


init = [ 0 , 0 ]
t_span = [ 0 , 30 ]
t_eval = np.linspace( t_span[0] , t_span[1] , 300 )

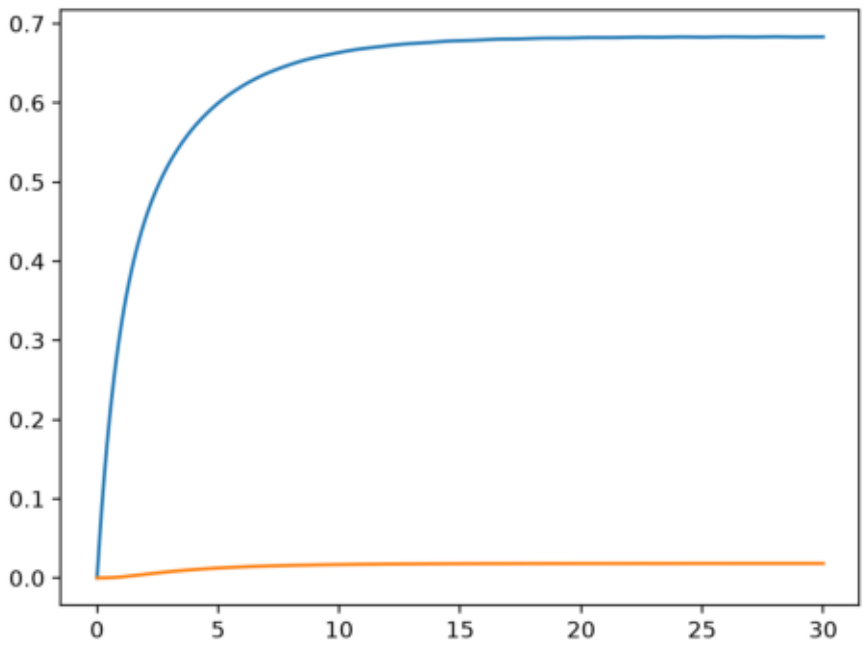
alpha = 0.1
beta = 10
rho = 25
x_out = 5

param = [ alpha , beta , rho , x_out ]

solution = solve_ivp( ozbudak_model, t_span, init , t_eval=t_eval , method='RK45', args=param )

TMG = solution.y[0]
LacY = solution.y[1]

plt.plot( solution.t , TMG )
plt.plot( solution.t , LacY )
plt.show()
```



演習2

```
import matplotlib.pyplot as plt
import numpy as np
from scipy.integrate import solve_ivp

def ozbudak_model( t , init , *param ):
    x = init[ 0 ]
    y = init[ 1 ]

    [ alpha , beta , rho , x_out ] = param

    dxdt = -x + alpha * x_out + beta * y
    dydt = (x ** 2) / (rho + (x ** 2)) - y

    return [ dxdt , dydt ]

def hysteresis_timeseries():
    init = [ 0 , 0 ]
    t_span = [ 0 , 60 ]

    alpha = 0.1
    beta = 10
    rho = 25
    x_out = 10

    param = [ alpha , beta , rho , x_out ]

    solution = solve_ivp( ozbudak_model, t_span, init , method='RK45', args=param )

    t = solution.t
    TMG = solution.y[0]
    LacY = solution.y[1]

    plt.plot( t , TMG )
    plt.plot( t , LacY )

    ##
    ## Repeat after elimitating TMG_out (x_out)
    ##

    x_init = TMG[-1]
    y_init = LacY[-1]
    init = [ x_init , y_init ]

    x_out = 0
    param = [ alpha , beta , rho , x_out ]

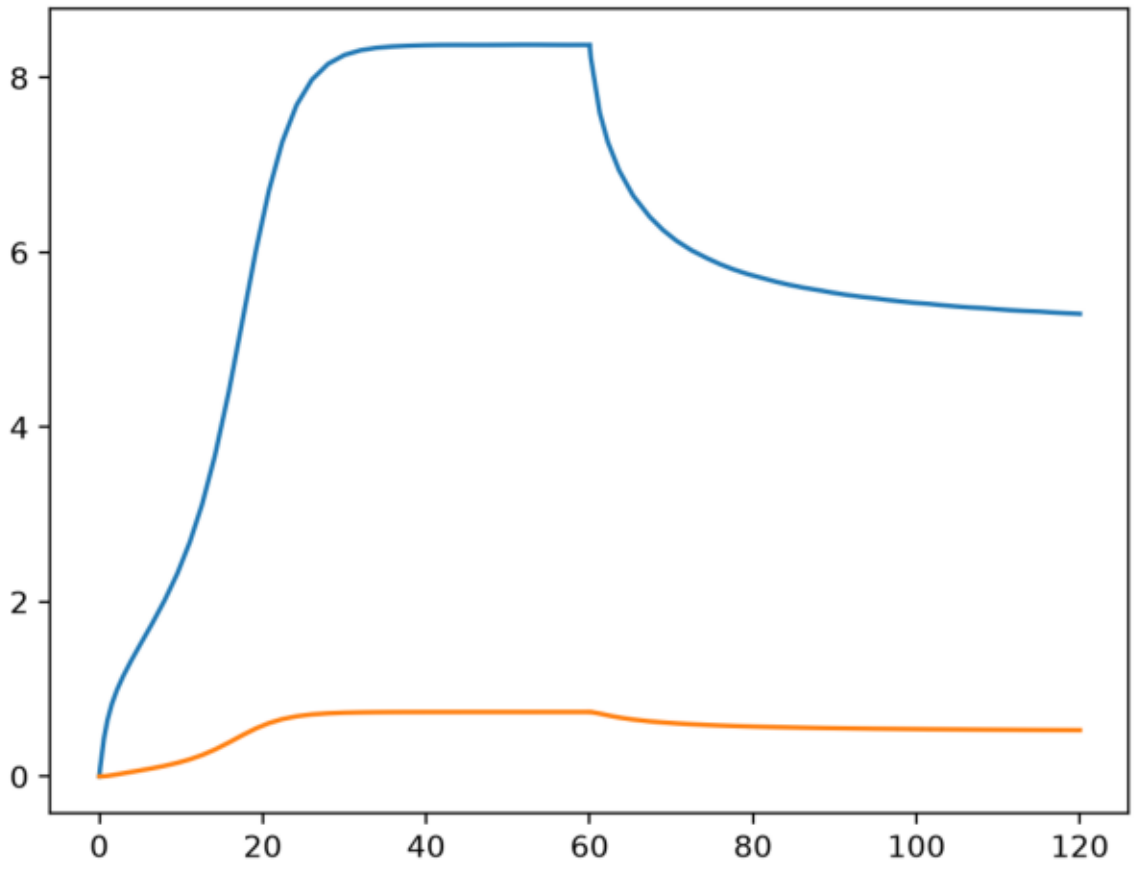
    t_span = [ 60 , 120 ]

    solution = solve_ivp( ozbudak_model, t_span, init , method='RK45', args=param )

    t = solution.t
    TMG = solution.y[0]
    LacY = solution.y[1]

    plt.plot( t , TMG , color='tab:blue')
    plt.plot( t , LacY , color='tab:orange')
    plt.show()

hysteresis_timeseries()
```


8
6
4
2
0
0
20
40
60
80
100
120

演習3

```
import matplotlib.pyplot as plt
import numpy as np
from scipy.integrate import solve_ivp

def ozbudak_model( t , init , *param ):
    x = init[ 0 ]
    y = init[ 1 ]

    [ alpha , beta , rho , x_out ] = param

    dxdt = -x + alpha * x_out + beta * y
    dydt =  x**2 / ( rho + x**2 ) - y

    return [ dxdt , dydt ]

def ozbudak_dose_response( x_out , y_init , marker ):
    init = [ 0 , y_init ]
    t_span = [ 0 , 60 ]

    alpha = 0.1
    beta = 10
    rho = 25

    param = [ alpha , beta , rho , x_out ]

    solution = solve_ivp( ozbudak_model, t_span, init , method='RK45', args=param )

    LacY = solution.y[1]
    y_end = LacY[-1]

    plt.plot( x_out , y_end , marker , fillstyle='none')

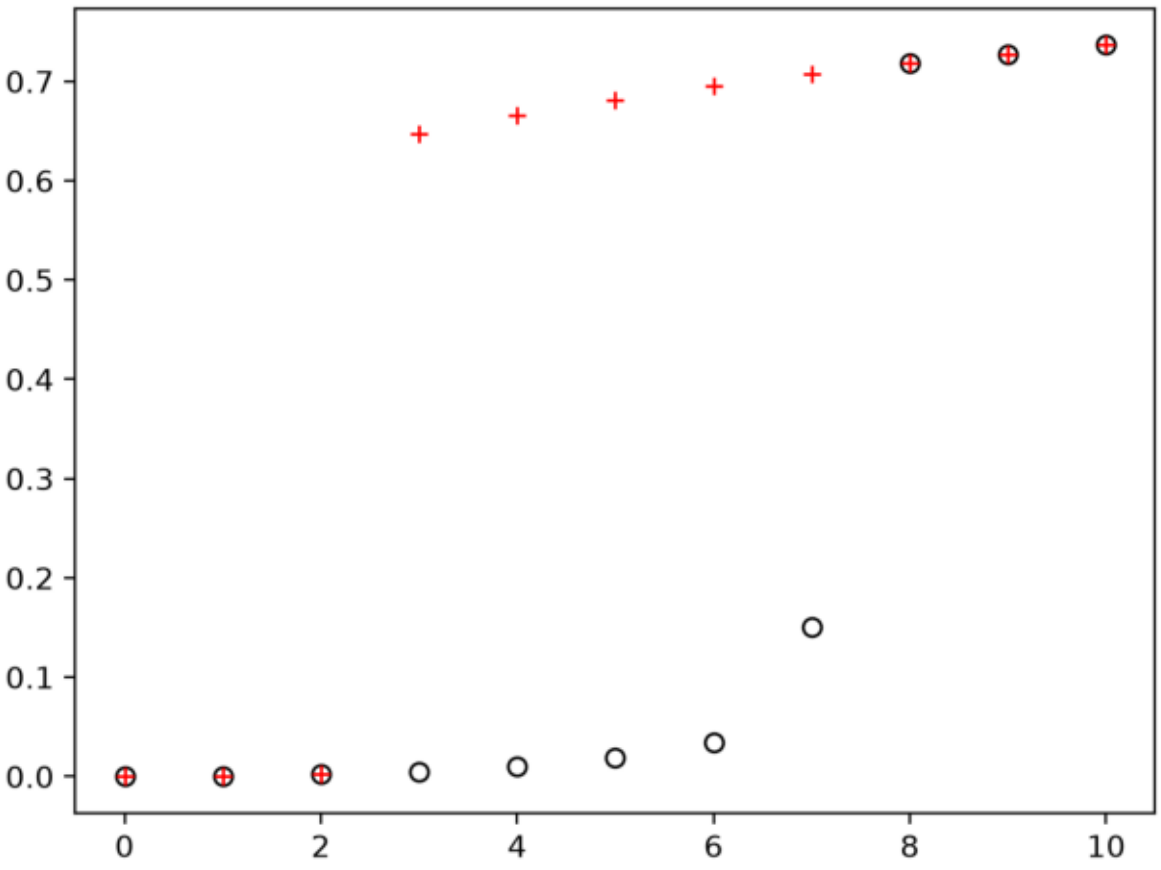
    return y_end

##
## Main
##

y_init = 0
for x_out in range(11):
    y_init = ozbudak_dose_response( x_out , y_init , 'ko' )

for x_out in reversed( range(11) ):
    y_init = ozbudak_dose_response( x_out, y_init, 'r+')

plt.show()
```


0.7
0.6
0.5
0.4
0.3
0.2
0.1
0.0
0
2
4
6
8
10

演習4

1. ポジティブ・フィードバックを遮断する改変

alphaがTMGの取り込み、betaがLacYによるTMGの取り込みであることを考慮する。このとき、dx/dtが$-x + \alpha X_{out} + \beta y$であり、
反応経路の図から、LacY(y) が TMG を取り込む → TMG (x) が増える → LacI の抑制が外れる → それにより LacY がさらに増加するというループがみられる。
このため、$+\beta y$が y から x に戻る項だと考えられる。

yがLacY-GFPからGFPに変わるとき、GFPはTMGを運ばないため、dx/dtの式におけるβyは効力を発揮しなくなる。そのため、以下のような微分方程式になるはずである：

```
dx/dt = -x + alpha * x_out
dy/dt = (x ** 2) / (rho + (x ** 2)) - y # LacIによる抑制と協同性の部分は不変のまま
```

2. シミュレーション

```
import matplotlib.pyplot as plt
import numpy as np
from scipy.integrate import solve_ivp

def ozbudak_model( t , init , *param ):
    x = init[ 0 ]
    y = init[ 1 ]

    [ alpha , beta , rho , x_out ] = param

    dxdt = -x + alpha * x_out # beta * y を撤去
    dydt =  x**2 / ( rho + x**2 ) - y

    return [ dxdt , dydt ]

def ozbudak_dose_response( x_out , y_init , marker ):
    init = [ 0 , y_init ]
    t_span = [ 0 , 60 ]

    alpha = 0.1
    beta = 10
    rho = 25

    param = [ alpha , beta , rho , x_out ]

    solution = solve_ivp( ozbudak_model, t_span, init , method='RK45', args=param )

    LacY = solution.y[1]
    y_end = LacY[-1]

    plt.plot( x_out , y_end , marker , fillstyle='none')

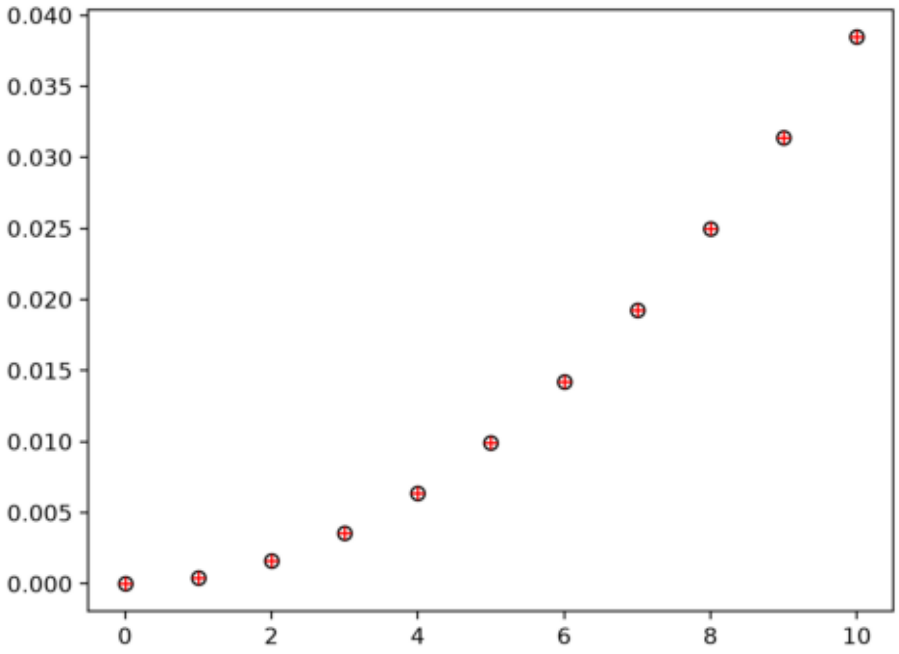
    return y_end

##
## Main
##
```

```
y_init = 0
for x_out in range(11):
    y_init = ozbudak_dose_response( x_out , y_init , 'ko' )

for x_out in reversed( range(11) ):
    y_init = ozbudak_dose_response( x_out, y_init, 'r+')

plt.show()
```



演習5

```
import matplotlib.pyplot as plt
import numpy as np
from scipy.integrate import solve_ivp

def ozbudak_model( t , init , *args ):
    x = init[ 0 ]
    y = init[ 1 ]

    [ alpha , beta , rho , x_out, n ] = args

    dxdt = -x + alpha * x_out + beta * y
    dydt =  x**n / ( rho + x**n ) - y

    return [ dxdt , dydt ]

def ozbudak_dose_response( x_out , y_init , marker, n ):
    init = [ 0 , y_init ]
    t_span = [ 0 , 60 ]

    alpha = 0.1
    beta = 10
    rho = 25

    solution = solve_ivp( ozbudak_model, t_span, init , method='RK45', args=[ alpha ,
                          beta , rho , x_out, n ] )

    LacY = solution.y[1]
    y_end = LacY[-1]

    plt.plot( x_out , y_end , marker , fillstyle='none')

    return y_end

##
## Main
##

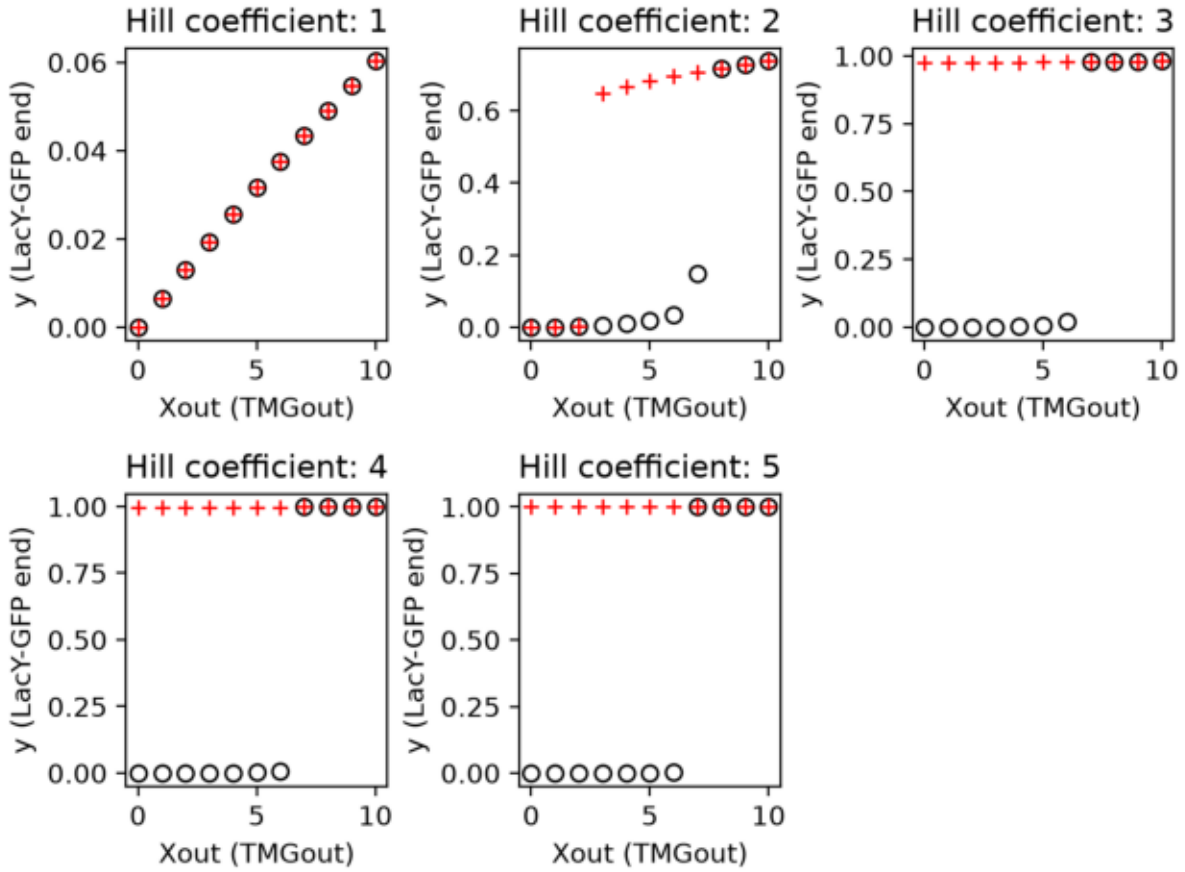
for n in range(1, 6):
    y_init = 0

    plt.subplot(2, 3, n)
    plt.title( f'Hill coefficient: {n}' )
    plt.xlabel( 'Xout (TMGout)' )
    plt.ylabel( 'y (LacY-GFP end)' )

    for x_out in range(11):
        y_init = ozbudak_dose_response( x_out , y_init , 'ko', n )

    for x_out in reversed( range(11) ):
        y_init = ozbudak_dose_response( x_out, y_init, 'r+', n )

plt.tight_layout()
plt.show()
```


Hill coefficient: 1
Hill coefficient: 2
Hill coefficient: 3
Hill coefficient: 4
Hill coefficient: 5
y (LacY-GFP end)
Xout (TMGout)
0.06
0.04
0.02
0.00
0.6
0.4
0.2
0.0
1.00
0.75
0.50
0.25
0
5
10